

Hex represented models

July 13, 2019

```
In [1]: from models import *
```

0.1 fit

```
In [2]: def fit(model,opt,epoch,dl,crit):
    for j in trange(epoch,desc="Epoch",position=1,mininterval=0):
        train_loss = 0
        val_loss = 0

        t=tqdm(iter(dl.trn_dl), leave=False, total=len(dl.trn_dl), miniters=0 ,postfix
        model.train()
        k = 0
        for data in t:
            k+=1
            t.close()
            t.set_postfix_str(f'loss : {float(train_loss)/k}',refresh = True)

            opt.zero_grad()

            data = data.type(torch.float).cuda()
            out = model(V(data))

            data = data.reshape(data.shape[0],data.shape[2],-1).type(torch.LongTensor)
            loss = crit(out,data)
            loss.backward()
            opt.step()

            train_loss += loss.data

        k = 0
        with torch.no_grad():
            model.eval()
            t=tqdm(iter(dl.val_dl), leave=False, total=len(dl.val_dl), miniters=0,postf
            for data in t:
                k+=1

                data = data.type(torch.float).cuda()
```

```

out = model(V(data))

data = data.reshape(data.shape[0],data.shape[2],-1).type(torch.LongTensor)

loss = crit(out,data)
val_loss += loss

t.close()
t.set_postfix_str(f'loss : {float(val_loss)/k}',refresh = True)

train_loss /= len(dl.trn_dl)
val_loss /= len(dl.val_dl)
model.record(val_loss)
print('train Loss: ', train_loss.tolist(), ' validation loss: ',
      val_loss.tolist())

```

```

In [3]: def fit(model,opt,epoch,dl,crit):
        for j in trange(epoch,desc="Epoch",position=1,mininterval=0):
            train_loss = 0
            val_loss = 0

            t=tqdm(iter(dl.trn_dl), leave=False, total=len(dl.trn_dl), miniters=0 ,postfix='')
            model.train()
            k = 0
            for data in t:
                k+=1
                t.close()

                opt.zero_grad()

                data = data.type(torch.float).cuda()
                out = model(V(data))

                data = data.reshape(data.shape[0],data.shape[2],-1).type(torch.LongTensor)

                loss = crit(out,data)

                loss.backward()
                opt.step()

                train_loss += loss.data
                t.set_postfix_str(f'loss : {float(train_loss)/k}',refresh = True)

            k = 0

```

```

with torch.no_grad():
    model.eval()
    t=tqdm(iter(dl.val_dl), leave=False, total=len(dl.val_dl), miniters=0,postt
    for data in t:
        k+=1

        data = data.type(torch.float).cuda()
        out = model(V(data))

        data = data.reshape(data.shape[0],data.shape[2],-1).type(torch.LongTensor)

        loss = crit(out,data)
        val_loss += loss

        t.close()
        t.set_postfix_str(f'loss : {float(val_loss)/k}',refresh = True)
    model.record(loss)
    train_loss /= len(dl.trn_dl)
    val_loss /= len(dl.val_dl)
    print('train Loss: ', train_loss.tolist(), ' validation loss: ',val_loss.t

```

0.1.1 Customized Dataset

```

In [124]: class dataset(Dataset):
    def __init__(self,base_dir , index ,dim, mode):
        super().__init__()

        files = os.listdir(base_dir)
        files = files

        names = [files[i] for i in index]
        self.data = [ base_dir +i for i in names]

        self.mode = mode

        self.dim = dim
        self.total_length = dim*dim
    def __getitem__(self, index):

        example = np.load(self.data[index]).astype(np.float)

        dim_example = len(example)
        diff = self.total_length - dim_example
        if self.mode == 1:

            left_pad = diff // 2
            right_pad = diff - left_pad

```

```

    else:
        left_pad = 0
        right_pad = diff
        example = np.pad(example, (left_pad, right_pad), 'constant').reshape(1, self.dim)

    return example

def __len__(self):
    return len(self.data)
    raise NotImplementedError

```

```
In [5]: a = nn.Embedding(16,16)
```

```
In [31]: a.weight[15]
```

```
Out[31]: tensor([ 0.5158, -0.3257,  1.4989,  0.9761, -0.4546,  1.1074, -0.0520,  0.2342,
                 -1.8671,  0.5426,  1.2566,  0.5186, -0.2628, -0.3428, -0.2006,  0.3009],
               grad_fn=<SelectBackward>)
```

```
In [32]: s = torch.zeros(3)
```

```
In [37]: s[:] = torch.ones(2)
```

```

-----

RuntimeError                                Traceback (most recent call last)

<ipython-input-37-43772be0b739> in <module>
----> 1 s[:] = torch.ones(2)

RuntimeError: The expanded size of the tensor (3) must match the existing size (2) at

```

```
In [35]: s
```

```
Out[35]: tensor([5., 5., 5.])
```

```
In [16]: a(torch.tensor([1]))
```

```
Out[16]: tensor([[ 0.2114, -0.8551,  1.1837, -0.5245, -0.7397, -0.9622,  1.5567,  0.6540,
                  0.5704, -1.1089, -0.1705,  1.2622,  0.1343,  1.4880,  2.1553,  0.0413]],
               grad_fn=<EmbeddingBackward>)
```

```
In [18]: torch.sigmoid(a(torch.tensor([0,1,0,5])).reshape(-1,32))
```

```
Out[18]: tensor([[0.2092, 0.6615, 0.7411, 0.2903, 0.6776, 0.6360, 0.6021, 0.1068, 0.8702,
                 0.3139, 0.5904, 0.4369, 0.4122, 0.4217, 0.5576, 0.6653, 0.5527, 0.2984,
                 0.7656, 0.3718, 0.3231, 0.2764, 0.8259, 0.6579, 0.6388, 0.2481, 0.4575,
```

```

0.7794, 0.5335, 0.8158, 0.8962, 0.5103],
[0.2092, 0.6615, 0.7411, 0.2903, 0.6776, 0.6360, 0.6021, 0.1068, 0.8702,
0.3139, 0.5904, 0.4369, 0.4122, 0.4217, 0.5576, 0.6653, 0.6266, 0.3287,
0.3548, 0.7007, 0.7109, 0.4379, 0.2503, 0.8207, 0.6106, 0.4636, 0.6670,
0.6364, 0.2149, 0.4382, 0.5402, 0.7553]], grad_fn=<SigmoidBackward>)

```

0.1.2 Split Data Train/Val

```
In [125]: def split(base_dir,dim,prc=0.2 , dataset_pct = 1,mode = 0):
```

```

    total_size = int(len(os.listdir(base_dir)) * dataset_pct)

    val_size = int(total_size*prc)

    trn_index = np.arange(total_size - val_size)
    val_index = np.arange(trn_index[-1]+1,total_size)

    val = dataset(base_dir , val_index,dim , mode = mode)
    trn = dataset(base_dir , trn_index,dim, mode = mode)
    return trn , val

```

```
In [126]: trn , val = split('dataset/hex/',dim=112 ,dataset_pct=0.12,mode= 0)
```

0.1.3 Prepare DataLoaders

```
In [127]: trn_dl = DataLoader(trn,batch_size=128,num_workers=2,shuffle=False)
          val_dl = DataLoader(val,batch_size=128,num_workers=2,shuffle=False)
```

```
In [128]: dl = CombinedDataLoader(trn_dl,val_dl)
```

0.2 Model

```
In [134]: class simple(nn.Module):
    def __init__(self):
        super().__init__()
        self.loss = []
        self.conv1 = nn.Conv2d(1,1,(8,14))
        self.pool = nn.MaxPool2d((30,30),stride=1)

        self.deconv1 =nn.ConvTranspose2d(1,1,(30,30))
        self.deconv2 = nn.ConvTranspose2d(1,16,(8,14))

    def forward(self,inputs):
        k = f.elu(self.conv1(inputs))
        k = self.pool(k)
        k = f.elu(self.deconv1(k))

        k = torch.softmax(self.deconv2(k),dim = 1)

```

```

        return k
    def record(self, record):
        self.loss.append(record)

```

```

In [135]: m = simple().cuda()
          opt = torch.optim.Adam(m.parameters(), 1e-3)

```

```

In [ ]: fit(m, opt, 6, dl, f.cross_entropy)
        #for i in opt.param_groups:
        #    i['lr'] = 1e-3
        #fit(m, 3)

```

```

HBox(children=(IntProgress(value=0, description='Epoch', max=6, style=ProgressStyle(description

```

```

100%|| 20/20 [00:04<00:00, 4.79it/s, loss : 2.0202598571777344]

```

```

train Loss: 2.419039487838745 validation loss: 2.0202598571777344

```

```

100%|| 20/20 [00:04<00:00, 4.80it/s, loss : 1.9446868896484375]

```

```

train Loss: 1.9697723388671875 validation loss: 1.9446868896484375

```

```

100%|| 20/20 [00:04<00:00, 4.75it/s, loss : 1.923896026611328]

```

```

train Loss: 1.933219313621521 validation loss: 1.923896074295044

```

```

48%|      | 38/80 [00:12<00:14, 2.97it/s, loss : 1.920909279271176]

```

```

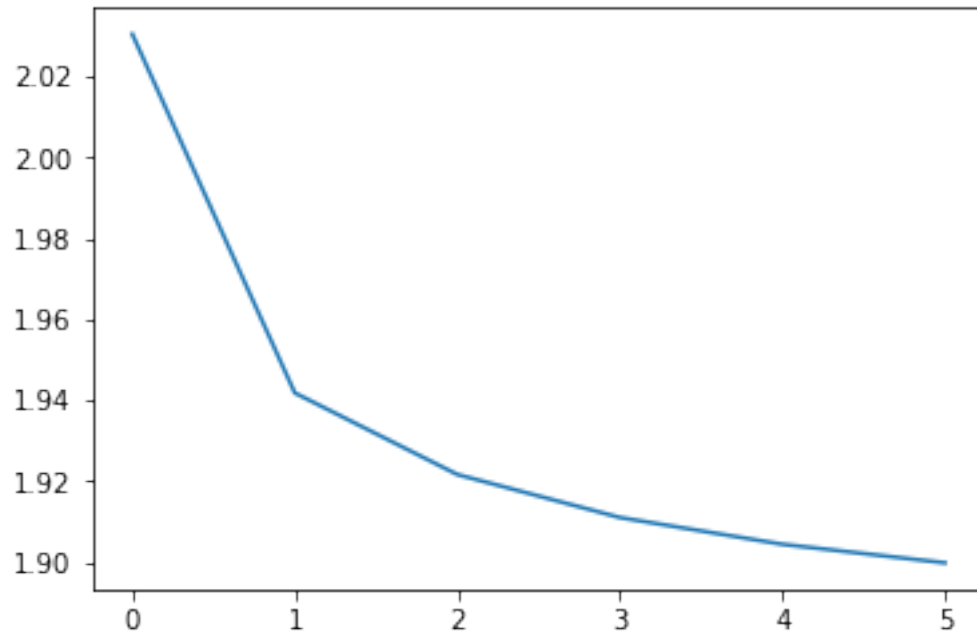
In [132]: plt.plot(m.loss)

```

```

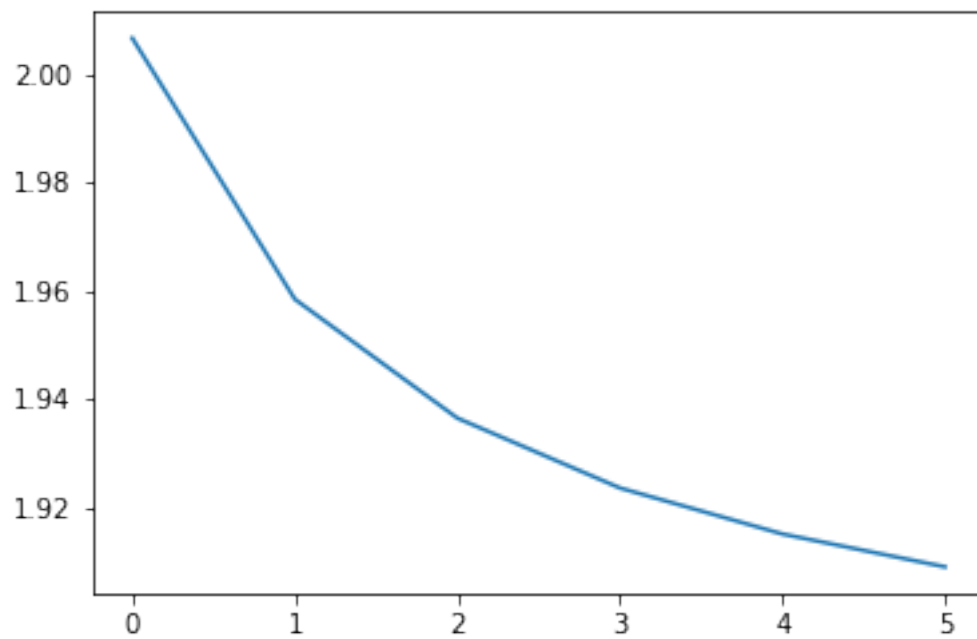
Out[132]: [<matplotlib.lines.Line2D at 0x7f07e8b035c0>]

```



```
In [87]: plt.plot(m.loss)
```

```
Out[87]: [<matplotlib.lines.Line2D at 0x7f07fe490fd0>]
```



```
In [88]: np.save('records/compressed/simple_l1.npy',m.loss)
```

0.3 Deeper Models

```
In [89]: class four_layer(nn.Module):
    def __init__(self):
        super().__init__()
        self.loss = []
        self.conv1 = nn.Conv2d(1,1,(32,32))
        self.pool = nn.MaxPool2d((16,16),stride=1)
        self.conv2 = nn.Conv2d(1,1,(8,8))
        self.pool2 = nn.AvgPool2d((8,8),stride=1)

        self.deconv1 =nn.ConvTranspose2d(1,1,(32,32))
        self.deconv2 = nn.ConvTranspose2d(1,1,(16,16))
        self.deconv3 =nn.ConvTranspose2d(1,1,(8,8))
        self.deconv4 = nn.ConvTranspose2d(1,16,(8,8))

    def forward(self,inputs):
        k = f.elu(self.conv1(inputs))
        k = self.pool(k)
        k = f.elu(self.conv2(k))
        k = self.pool2(k)

        k = f.elu(self.deconv1(k))
        k = f.elu(self.deconv2(k))
        k = f.elu(self.deconv3(k))
        k = torch.softmax(self.deconv4(k),dim = 1)

        return k
    def record(self,record):
        self.loss.append(record)
```

```
In [90]: m = four_layer().cuda()
        opt = torch.optim.Adam(m.parameters(),1e-3)
```

```
In [91]: a = fit(m,opt,6,dl,f.cross_entropy)
        #for i in opt.param_groups:
        #    i['lr'] = 1e-3
        #fit(m,3)
```

HBox(children=(IntProgress(value=0, description='Epoch', max=6, style=ProgressStyle(description

100%|| 20/20 [00:03<00:00, 5.88it/s, loss : 1.9793943405151366]

train Loss: 2.29752516746521 validation loss: 1.9793943166732788

100%|| 20/20 [00:03<00:00, 5.90it/s, loss : 1.9370372772216797]

```
train Loss: 1.9551786184310913    validation loss: 1.9370373487472534
```

```
100%|| 20/20 [00:03<00:00, 5.90it/s, loss : 1.9186023712158202]
```

```
train Loss: 1.9268721342086792    validation loss: 1.9186023473739624
```

```
100%|| 20/20 [00:03<00:00, 5.89it/s, loss : 1.9084833145141602]
```

```
train Loss: 1.9133005142211914    validation loss: 1.9084833860397339
```

```
100%|| 20/20 [00:03<00:00, 5.52it/s, loss : 1.9013990402221679]
```

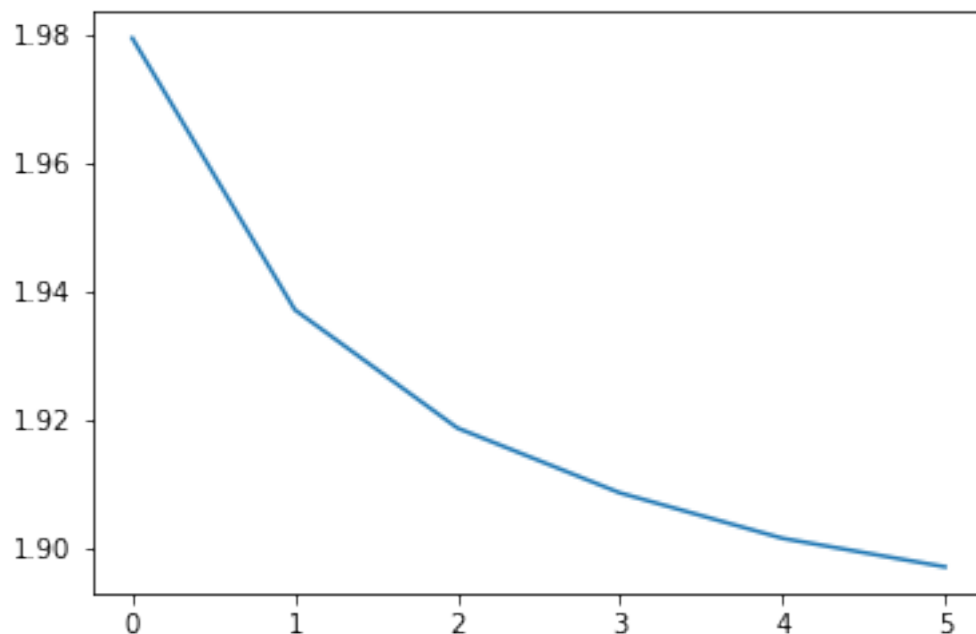
```
train Loss: 1.9047945737838745    validation loss: 1.90139901638031
```

```
100%|| 20/20 [00:03<00:00, 5.84it/s, loss : 1.896951675415039]
```

```
train Loss: 1.8990249633789062    validation loss: 1.896951675415039
```

```
In [92]: plt.plot(m.loss)
```

```
Out[92]: [<matplotlib.lines.Line2D at 0x7f07f85e6080>]
```



```
In [93]: np.save('records/compressed/f_1_1_l1.npy',m.loss)
```

Start with max Pooling

```
In [94]: class fl_max_norm(nn.Module):
        def __init__(self):
            super().__init__()
            self.loss = []
            self.conv1 = nn.Conv2d(1,1,(32,32))
            self.pool = nn.MaxPool2d((16,16),stride=1)
            self.conv2 = nn.Conv2d(1,1,(8,8))
            self.pool2 = nn.AvgPool2d((8,8),stride=1)

            self.deconv1 =nn.ConvTranspose2d(1,1,(32,32))
            self.deconv2 = nn.ConvTranspose2d(1,1,(16,16))
            self.deconv3 =nn.ConvTranspose2d(1,1,(8,8))
            self.deconv4 = nn.ConvTranspose2d(1,16,(8,8))

        def forward(self,inputs):
            k = self.pool(inputs)
            k = f.normalize(f.elu(self.conv1(k)))
            k = f.elu(self.conv2(k))
            k = self.pool2(k)

            k = f.elu(self.deconv1(k))
            k = f.elu(self.deconv2(k))
            k = f.elu(self.deconv3(k))
            k = torch.softmax(self.deconv4(k),dim = 1)

            return k
        def record(self,record):
            self.loss.append(record)
```

```
In [95]: m = fl_max_norm().cuda()
        opt = torch.optim.Adam(m.parameters(),1e-3)
```

```
In [96]: fit(m,opt,6,dl,f.cross_entropy)
        #for i in opt.param_groups:
        #    i['lr'] = 1e-3
        #fit(m,3)
```

HBox(children=(IntProgress(value=0, description='Epoch', max=6, style=ProgressStyle(description

100%|| 20/20 [00:03<00:00, 6.00it/s, loss : 2.0350927352905273]

train Loss: 2.277134418487549 validation loss: 2.035092830657959

100%|| 20/20 [00:03<00:00, 6.05it/s, loss : 1.9674306869506837]

```
train Loss: 2.1335837841033936    validation loss: 1.9674307107925415
```

```
100%|| 20/20 [00:03<00:00, 6.06it/s, loss : 1.938957405090332]
```

```
train Loss: 1.9515100717544556    validation loss: 1.9389574527740479
```

```
100%|| 20/20 [00:03<00:00, 6.06it/s, loss : 1.9236507415771484]
```

```
train Loss: 1.9307512044906616    validation loss: 1.9236507415771484
```

```
100%|| 20/20 [00:03<00:00, 6.08it/s, loss : 1.9141813278198243]
```

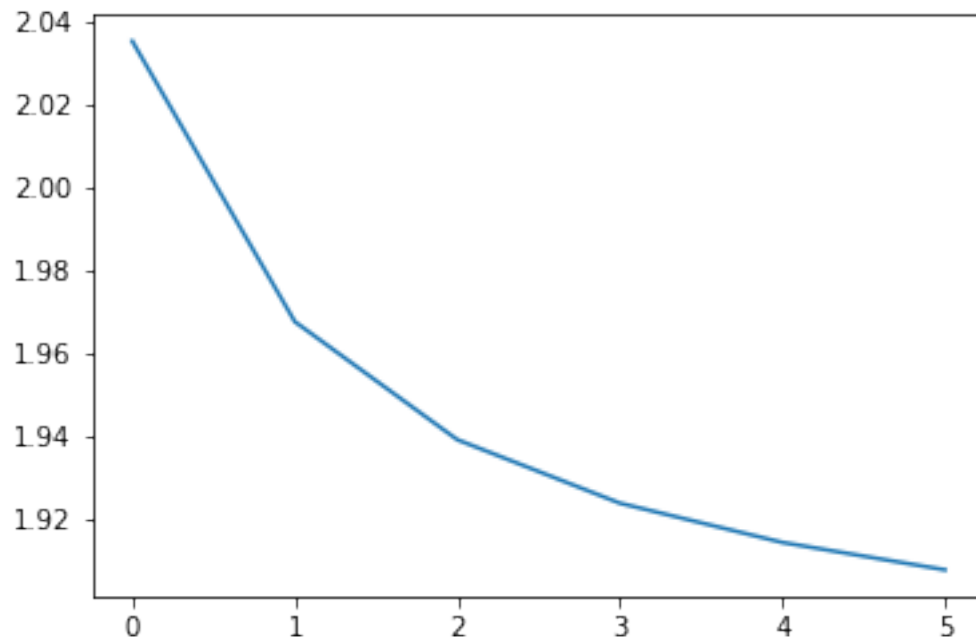
```
train Loss: 1.9186344146728516    validation loss: 1.9141813516616821
```

```
100%|| 20/20 [00:03<00:00, 6.07it/s, loss : 1.907576560974121]
```

```
train Loss: 1.9107637405395508    validation loss: 1.907576560974121
```

```
In [97]: plt.plot(m.loss)
```

```
Out[97]: [<matplotlib.lines.Line2D at 0x7f07fcd64470>]
```



```
In [98]: np.save('records/compressed/f_l_1_Max_Norm_l1.npy',m.loss)
```

1 Mode 2

```
In [99]: trn , val = split('dataset/hex/',dim=112 ,dataset_pct=0.12,mode= 1)
```

1.0.1 Prepare DataLoaders

```
In [100]: trn_dl = DataLoader(trn,batch_size=128,num_workers=2,shuffle=False)
          val_dl = DataLoader(val,batch_size=128,num_workers=2,shuffle=False)
```

```
In [101]: dl = CombinedDataLoader(trn_dl,val_dl)
```

1.1 Model

```
In [102]: class simple(nn.Module):
          def __init__(self):
              super().__init__()
              self.loss = []
              self.conv1 = nn.Conv2d(1,1,(8,14))
              self.pool = nn.MaxPool2d((30,30),stride=1)

              self.deconv1 =nn.ConvTranspose2d(1,1,(30,30))
              self.deconv2 = nn.ConvTranspose2d(1,16,(8,14))

          def forward(self,inputs):
              k = f.elu(self.conv1(inputs))
              k = self.pool(k)
              k = f.elu(self.deconv1(k))

              k = torch.softmax(self.deconv2(k),dim = 1)

              return k
          def record(self,record):
              self.loss.append(record)
```

```
In [103]: m = simple().cuda()
          opt = torch.optim.Adam(m.parameters(),1e-3)
```

```
In [104]: fit(m,opt,6,dl,f.cross_entropy)
          #for i in opt.param_groups:
          #    i['lr'] = 1e-3
          #fit(m,3)
```

HBox(children=(IntProgress(value=0, description='Epoch', max=6, style=ProgressStyle(description

100%|| 20/20 [00:04<00:00, 4.77it/s, loss : 2.0168041229248046]

train Loss: 2.241689443588257 validation loss: 2.0168042182922363

100%|| 20/20 [00:04<00:00, 4.77it/s, loss : 1.9702579498291015]

train Loss: 1.990351915359497 validation loss: 1.9702579975128174

100%|| 20/20 [00:04<00:00, 4.77it/s, loss : 1.9472803115844726]

train Loss: 1.9579137563705444 validation loss: 1.9472802877426147

100%|| 20/20 [00:04<00:00, 4.76it/s, loss : 1.9330829620361327]

train Loss: 1.9398040771484375 validation loss: 1.933082938194275

100%|| 20/20 [00:04<00:00, 4.76it/s, loss : 1.9233528137207032]

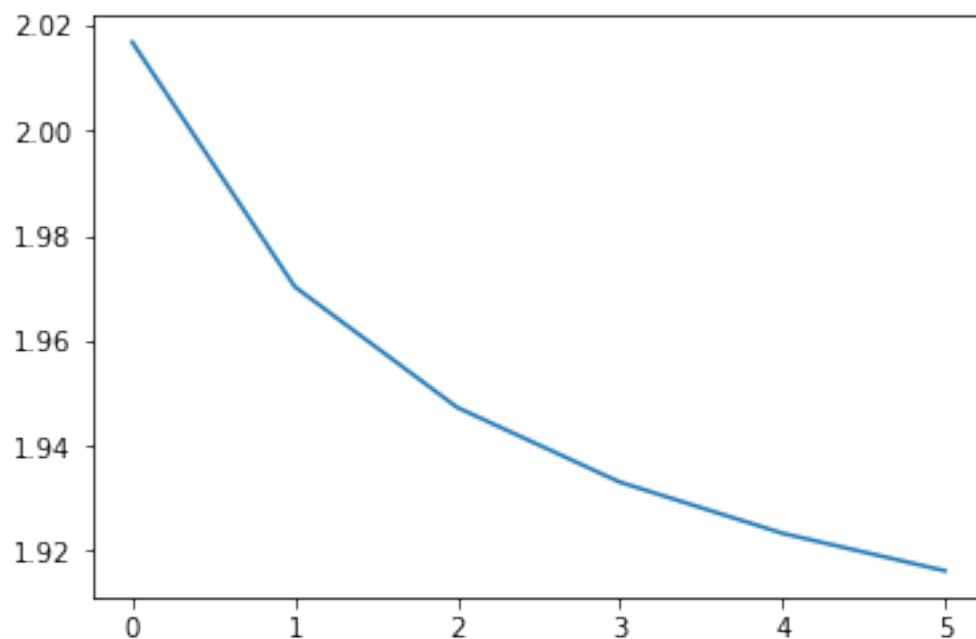
train Loss: 1.928030014038086 validation loss: 1.923352837562561

100%|| 20/20 [00:04<00:00, 4.76it/s, loss : 1.9162092208862305]

train Loss: 1.9196760654449463 validation loss: 1.9162092208862305

In [105]: plt.plot(m.loss)

Out[105]: [<matplotlib.lines.Line2D at 0x7f07f84051d0>]



In [106]: np.save('records/compressed/simple_l1_center.npy',m.loss)

1.2 Deeper Models

```
In [107]: class four_layer(nn.Module):
    def __init__(self):
        super().__init__()
        self.loss = []
        self.conv1 = nn.Conv2d(1,1,(32,32))
        self.pool = nn.MaxPool2d((16,16),stride=1)
        self.conv2 = nn.Conv2d(1,1,(8,8))
        self.pool2 = nn.AvgPool2d((8,8),stride=1)

        self.deconv1 = nn.ConvTranspose2d(1,1,(32,32))
        self.deconv2 = nn.ConvTranspose2d(1,1,(16,16))
        self.deconv3 = nn.ConvTranspose2d(1,1,(8,8))
        self.deconv4 = nn.ConvTranspose2d(1,16,(8,8))

    def forward(self,inputs):
        k = f.elu(self.conv1(inputs))
        k = self.pool(k)
        k = f.elu(self.conv2(k))
        k = self.pool2(k)

        k = f.elu(self.deconv1(k))
        k = f.elu(self.deconv2(k))
        k = f.elu(self.deconv3(k))
        k = torch.softmax(self.deconv4(k),dim = 1)

        return k
    def record(self,record):
        self.loss.append(record)
```

```
In [108]: m = four_layer().cuda()
    opt = torch.optim.Adam(m.parameters(),1e-3)
```

```
In [109]: a = fit(m,opt,6,dl,f.cross_entropy)
    #for i in opt.param_groups:
    #    i['lr'] = 1e-3
    #fit(m,3)
```

HBox(children=(IntProgress(value=0, description='Epoch', max=6, style=ProgressStyle(description

100%|| 20/20 [00:03<00:00, 5.88it/s, loss : 2.0061683654785156]

train Loss: 2.2617194652557373 validation loss: 2.0061683654785156

100%|| 20/20 [00:03<00:00, 5.88it/s, loss : 1.9676191329956054]

train Loss: 1.9839963912963867 validation loss: 1.9676191806793213

100%|| 20/20 [00:03<00:00, 5.65it/s, loss : 1.9485004425048829]

train Loss: 1.9574100971221924 validation loss: 1.9485005140304565

100%|| 20/20 [00:03<00:00, 5.90it/s, loss : 1.9359352111816406]

train Loss: 1.9419279098510742 validation loss: 1.9359352588653564

100%|| 20/20 [00:03<00:00, 5.91it/s, loss : 1.926877212524414]

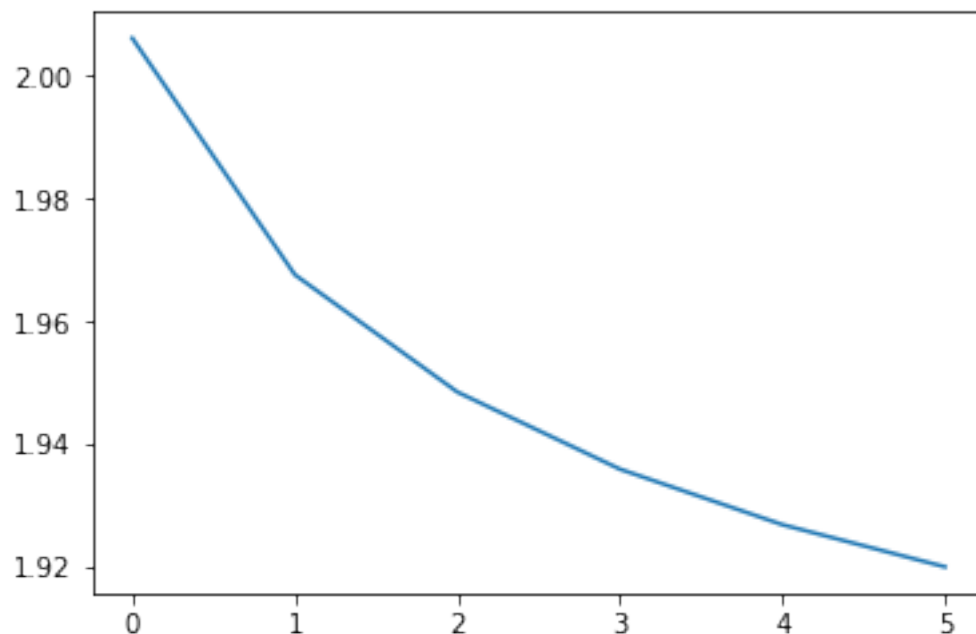
train Loss: 1.9312450885772705 validation loss: 1.9268772602081299

100%|| 20/20 [00:03<00:00, 5.91it/s, loss : 1.9200117111206054]

train Loss: 1.9233486652374268 validation loss: 1.9200117588043213

In [110]: plt.plot(m.loss)

Out[110]: [<matplotlib.lines.Line2D at 0x7f07f85fb278>]



In [111]: np.save('records/compressed/f_1_1_l1_center.npy',m.loss)

Start with max Pooling

```
In [112]: class fl_max_norm(nn.Module):
    def __init__(self):
        super().__init__()
        self.loss = []
        self.conv1 = nn.Conv2d(1,1,(32,32))
        self.pool = nn.MaxPool2d((16,16),stride=1)
        self.conv2 = nn.Conv2d(1,1,(8,8))
        self.pool2 = nn.AvgPool2d((8,8),stride=1)

        self.deconv1 =nn.ConvTranspose2d(1,1,(32,32))
        self.deconv2 = nn.ConvTranspose2d(1,1,(16,16))
        self.deconv3 =nn.ConvTranspose2d(1,1,(8,8))
        self.deconv4 = nn.ConvTranspose2d(1,16,(8,8))

    def forward(self,inputs):
        k = self.pool(inputs)
        k = f.normalize(f.elu(self.conv1(k)))
        k = f.elu(self.conv2(k))
        k = self.pool2(k)

        k = f.elu(self.deconv1(k))
        k = f.elu(self.deconv2(k))
        k = f.elu(self.deconv3(k))
        k = torch.softmax(self.deconv4(k),dim = 1)

        return k
    def record(self,record):
        self.loss.append(record)
```

```
In [113]: m = fl_max_norm().cuda()
    opt = torch.optim.Adam(m.parameters(),1e-3)
```

```
In [114]: fit(m,opt,6,dl,f.cross_entropy)
    #for i in opt.param_groups:
    #    i['lr'] = 1e-3
    #fit(m,3)
```

HBox(children=(IntProgress(value=0, description='Epoch', max=6, style=ProgressStyle(description

100%|| 20/20 [00:03<00:00, 6.05it/s, loss : 2.0146306991577148]

train Loss: 2.480337619781494 validation loss: 2.0146307945251465

100%|| 20/20 [00:03<00:00, 6.06it/s, loss : 1.9602531433105468]

train Loss: 1.9815059900283813 validation loss: 1.960253119468689

100%|| 20/20 [00:03<00:00, 5.99it/s, loss : 1.938268280029297]

train Loss: 1.9483288526535034 validation loss: 1.9382683038711548

100%|| 20/20 [00:03<00:00, 6.07it/s, loss : 1.925115203857422]

train Loss: 1.9312915802001953 validation loss: 1.9251152276992798

100%|| 20/20 [00:03<00:00, 6.07it/s, loss : 1.916312026977539]

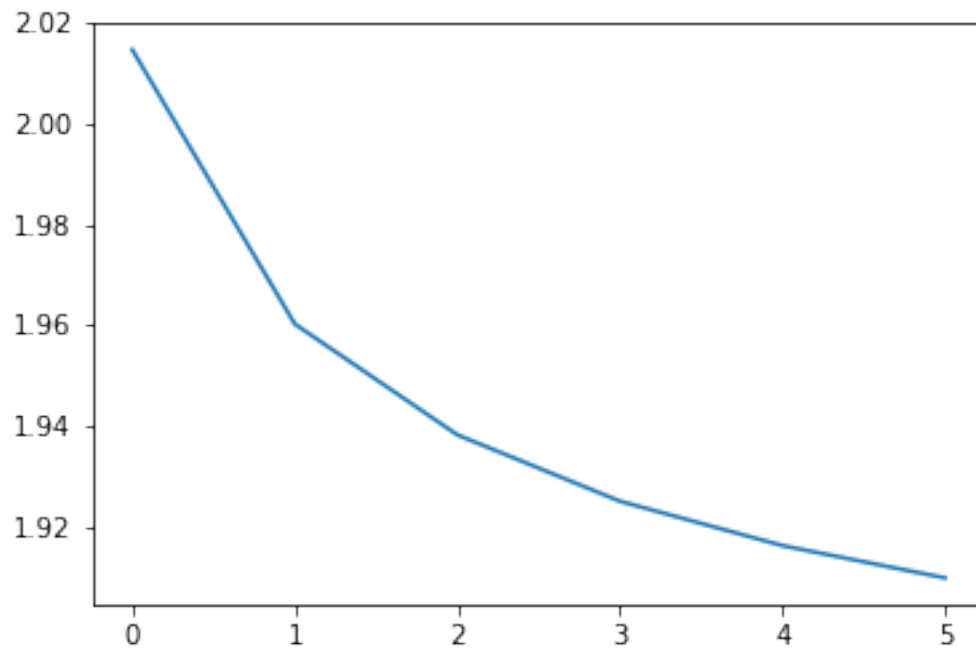
train Loss: 1.9205169677734375 validation loss: 1.9163120985031128

100%|| 20/20 [00:03<00:00, 6.07it/s, loss : 1.9099037170410156]

train Loss: 1.913002848625183 validation loss: 1.9099037647247314

In [115]: plt.plot(m.loss)

Out[115]: [<matplotlib.lines.Line2D at 0x7f07f8507c18>]

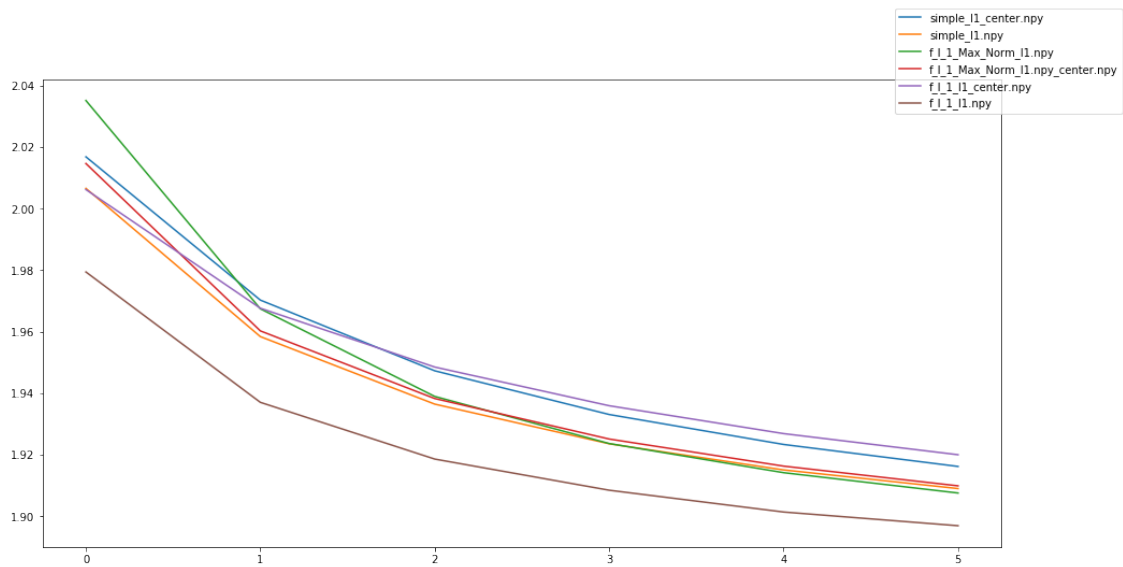


```
In [116]: np.save('records/compressed/f_l_1_Max_Norm_l1.npy_center',m.loss)
```

```
In [120]: a = os.listdir('records/compressed/cross/')
```

```
In [121]: fig , ax = plt.subplots(figsize=(16,8))
          for i in a :
              b = np.load('records/compressed/cross/'+i)
              ax.plot(b, label=i)
          fig.legend()
```

```
Out[121]: <matplotlib.legend.Legend at 0x7f07f8054470>
```



Left padding looks to achieve better results